

Reviewer Pack — Free Entropy Gap in Read-Twice Branching Programs

Entry 4eca2ec83f86 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 11:34:26 UTC

Free Entropy Gap in Read-Twice Branching Programs

Entry ID: 4eca2ec83f86 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 11:34:26 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

Let P be a read-twice branching program over n variables. The free entropy $\phi(P)$ of its transition matrix satisfies $\phi(P) \geq \log^2 n$, while read-once BPs have $\phi(P) \leq \log n$. For all P with size $\geq 2^{n/2}$, $\phi(P) \geq \Omega(\log^2 n)$ holds.

Rationale (proposer's reasoning):

Free entropy captures non-commutative randomness in transition matrices. Read-twice BPs exhibit structured dependencies that amplify spectral complexity, creating a gap detectable via free probability invariants. This mirrors the separation between read-once and read-many BPs in algebraic complexity.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1a1b0187f6457f6c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10 # Start with a small size and increase if needed

    while True:
        P = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]

        # Check if the transition matrix is valid
        if all(all(P[i][j] == P[j][i] for j in range(i+1, n)) for i in range(n)):
            break

    # Compute the free entropy  $\phi(P)$ 
    def log_moment_generating_function(matrix):
        n = len(matrix)
        moment = 0
        for k in range(1, n + 1):
            sum_k = 0
            for i in range(k):
                for j in range(k):
                    if i != j:
                        sum_k += matrix[i][j] * matrix[j][i]
            moment += math.log((n - k) / (k * (n - k))) * sum_k
        return moment

     $\phi_P$  = log_moment_generating_function(P)

    # Check the size condition and the free entropy threshold
    size_condition = 2 ** n // 2 <= len(P)
    free_entropy_threshold = math.log(n) ** 2

    result = {
        "metric_name": "free_entropy",
        "metric_value":  $\phi_P$ ,
        "instances_tested": 1,
        "conjecture_holds": size_condition and  $\phi_P$  >= free_entropy_threshold,
        "counterexample": "" if size_condition and  $\phi_P$  >= free_entropy_threshold else f"Size: {len(P)}"
    }
```

```

    return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)] # Default to first

    results = []
    total_metric_value = 0.0
    support_count = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        total_metric_value += trial_result["metric_value"]
        if trial_result["conjecture_holds"]:
            support_count += 1

    results.append(trial_result)

    mean_metric_value = total_metric_value / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = support_count / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"Size condition or free entropy threshold not me
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with stricter resource limits

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	53886
2	preregistration	ollama_remote	qwen3:8b	0	0	27786
3	novelty	ollama_remote	qwen3:8b	0	0	24084
4	novelty	ollama_remote	qwen3:8b	0	0	16938
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16411
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12553
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11235
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8314
9	judge	ollama_remote	qwen3:8b	0	0	18038

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 189245 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4eca2ec83f86.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4eca2ec83f86.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4eca2ec83f86.tar.gz` (if generated)